Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers

Ahmed Gouda Ismail[1*], Mohamed S. Mohamed[1] and Tarek Ahmed Mahmoud[2]

[1*]Computer Engineering and Artificial Intelligence Department, Military Technical College, Cairo, Egypt.

[2]Computer Engineering Department, Egypt University of Informatics, Cairo, Egypt.

*Corresponding author(s). E-mail(s): a7medgouda1@gmail.com;
Contributing authors: mohamedms@mtc.edu.eg; tarek.mohamed@eui.edu.eg;

Abstract

Multi-object tracking pipelines are often built from a detector and a tracker developed separately. When the detector is replaced, retrained, or quantized for embedded hardware, its confidence scale changes, while the tracker keeps thresholds tuned for another detector. This paper presents a training-free self-calibrating control layer (SCL) that sits between a frozen detector and a frozen tracker. It calibrates itself online: it splits the detector scores of recent frames with an automatic histogram threshold (Otsu's method) and decides whether the score stream is clean or noisy. On clean streams it keeps the tracker's operating point. On noisy streams it selects the candidates that reach the tracker, remaps their scores, and sets the tracker's thresholds. It also gives a continuation stage to trackers that lack one. It needs no labels and uses no future frames. One frozen configuration was evaluated on seven published trackers, four detection sources, and two datasets with paired bootstrap intervals. It raised the higher order tracking accuracy (HOTA) of two single-stage trackers by 0.61 points, with intervals above zero: OC-SORT, used during development, and PD-SORT, which was not. When all detector scores were multiplied by 0.5, two trackers found no detection above their thresholds and scored 0 HOTA; with SCL, both reached about 66 HOTA. With a noisy RT-DETR-L detector, it raised the multi-object tracking accuracy (MOTA) of ByteTrack and BoT-SORT by 6.0 and 7.9 points. On four trackers with a low-score stage and well-matched detections, HOTA changed by at most 0.06 points. SCL takes 2.95 ms per frame on a four-core CPU without a GPU, so it adds very little latency and is suitable for real-time use.

Keywords: Multi-object tracking, Tracking by detection, Detector confidence calibration, Online adaptation, Otsu thresholding, Real-time image processing

1 Introduction

Multi-object tracking (MOT) finds every object of interest in a video and keeps the same identity for each object over time. It is a core part of surveillance, driving, and airborne systems [1–3], and many of these systems must produce results with low delay. Most practical trackers follow the tracking-by-detection design. A detector scores candidate boxes in each frame, and the tracker links the boxes whose scores pass its thresholds into trajectories [4–6].

In deployed systems the detector and the tracker are rarely developed together. A detector is replaced by a faster model, retrained for a new sensor, or quantized for embedded hardware. The tracker, however, keeps the thresholds that its authors tuned for another detector. Detector confidence is not calibrated across models and conditions [7], so the new detector usually has a different score scale, and nothing in the pipeline notices the change.

The effect can be large. In the experiments of this paper, halving the scores of a published detector drives two well-known trackers to 0 higher order tracking accuracy (HOTA) [8], because no detection reaches their fixed thresholds (Sect. 7.3). Retuning every tracker for every detector needs labelled data from the deployment domain, which an operational system usually does not have.

Existing methods only partly address this problem. Adaptive-threshold trackers adjust their own thresholds from the detections of each frame, but each method is built into one specific tracker [9–11]. Confidence calibration methods correct the detector scores, but they need labelled data [7]. Adaptive inference methods learn or profile a policy for one model [12, 13]. What is missing is a training-free component that works outside the tracker, calibrates itself from the detector's own score stream, and leaves the tracker unchanged when the tracker already fits that stream.

This paper presents such a component: a self-calibrating control layer (SCL) between a frozen detector and a frozen tracker. It reads the stream of detector scores and the operating point that the tracker declares, and it decides online whether the tracker needs help. The central idea is that the detector's own scores tell whether its output fits the tracker. SCL looks at the score distribution of recent frames and asks how cleanly it separates confident objects from clutter. If the separation is clean, the tracker keeps its own operating point, and the layer only adds help that the tracker lacks, such as a stage that continues tracks. If it is not, SCL chooses which candidates reach the tracker, remaps their scores, and sets the tracker's thresholds. What is new is therefore a layer outside the tracker that needs no labels, uses one frozen configuration for many trackers, and stays out of the way when the tracker already fits the detector.

The main contributions are:

1. SCL, a training-free control layer that calibrates itself from the detector score stream, uses no future frames, keeps the tracker's operating point when the stream is clean, and intervenes fully only when it is noisy (Sect. 4).
2. A host contract of four numbers that lets one frozen configuration work with single-stage and two-stage trackers without modifying them (Sect. 4.5).
3. A development path from always-on banding (AOB), a prior design that always intervened, in which each step is compared with that baseline (Sect. 5).
4. An evaluation of one frozen configuration on seven published trackers, four detection sources, and two datasets, with development and predeclared external roles, paired bootstrap intervals, and a latency measurement on a CPU (Sects. 6 and 7).

The rest of the paper is organized as follows. Section 2 describes the data, evaluation roles, and metrics, and Section 3 reviews related work. Section 4 specifies SCL with all its constants, and Section 5 explains how it was developed from an earlier design that always intervened. The configuration was then frozen before any external system was run. Section 6 gives the experimental setup, and Section 7 the results and their analysis. Section 8 concludes the paper.

2 Data and Evaluation Protocol

2.1 Detectors and data

Table 1 lists the evaluation cells, that is, each combination of data, detector, and tracker (the trackers are introduced in Sect. 6). On MOT17 val-half [14] the trackers use the published YOLOX-X detections [15]. The emission floor, the lowest detector score passed on, is 0.01 or 0.1 (Table 8). On KITTI tracking training [16], the trackers run with two detectors that differ from the MOT17 detector: YOLOv8n [17] and RT-DETR-L [18]. To isolate the calibration problem, the published MOT17 detections were also transformed before the tracker by four monotone maps: cubing (s^3), halving ($0.5\,s$), and logit temperatures $T = 2$ and

$T = 0.5$, $\sigma(z/T)$. The trackers kept their published thresholds. The external systems use the authors' released detections on MOT17 val-half.

2.2 Evaluation roles and metrics

Every cell has one of two roles (Table 1). Development cells were visible while SCL was designed: MOT17 with four trackers, KITTI with two other detectors, and the score transforms. The predeclared external systems, PD-SORT and Hybrid-SORT, were selected and declared before they were run, and they were run once with the frozen configuration. For each external host, the reproduction against the authors' number is classed as EXACT (within 0.2 HOTA) or CLOSE (within 1.0 HOTA with a documented, untuned cause).

Table 2 gives the three tracking metrics and their definitions. They are computed with the TrackEval library (commit 12c8791) and the official ground truth of each benchmark, following MOTChallenge practice [19]. On KITTI tracking, HOTA is the official value averaged over cars and pedestrians.

All comparisons are paired. Differences are pooled over sequences, and their 95% intervals come from a percentile bootstrap over sequences [22] with 10,000 resamples and seed 42. A difference is called significant when its interval excludes zero. As a distribution-free complement, per-sequence wins, ties ($|\Delta\text{HOTA}| < 0.01$), and losses are also counted.

3 Related Work

SCL touches four research directions: the trackers it serves, the confidence of the detectors that feed them, trackers that adapt their own thresholds, and methods that adapt inference at run time. They are reviewed below in this order, and Table 3 compares the closest methods with SCL.

Most practical trackers follow the tracking-by-detection design. SORT predicts each box with a Kalman filter and matches the predictions to new detections by overlap with the Hungarian method [4]. ByteTrack introduced a second association stage that uses low-confidence detections to continue existing tracks [5], and many later trackers kept this two-stage design. Others, such as OC-SORT [6], use a single confidence threshold. All of these trackers read detector scores through fixed thresholds that were tuned for one detector, usually YOLOX [15]. Seven published trackers of both kinds are used in this paper as hosts and are kept unchanged (Sect. 6).

These fixed thresholds are a weak point, because detector scores do not carry the same meaning across models. Detectors of different families, such as YOLOX [15], YOLOv8 [17], YOLOv9 [23], DETR [24], and RT-DETR [18], produce different score distributions. Their confidence is not calibrated across models and conditions, and calibration methods for object detection fit a mapping on labelled data [7]. A tracker tuned for one detector therefore cannot assume that another detector's scores mean the same thing, and labels to recalibrate them are rarely available after deployment. A label-free tool for this exists in image processing: Otsu's method selects a threshold from a histogram by maximizing the separation between two classes [25]. SCL applies it to the logits (log-odds) of the detector scores of recent frames.

Some trackers already adapt their own confidence thresholds. Adaptive ByteTrack derives the confidence threshold of ByteTrack from the detections of each frame [9]. AdapTrack adapts the thresholds of its matching step [10]. ACT sets the high and low thresholds of each frame from the first quartile of the confidence scores and the number of candidates [11]. These methods are built into one tracker and use statistics of the current frame. SCL differs in four ways. Its statistics come from past frames. It decides for the whole stream whether to intervene at all, and otherwise keeps the tracker's own operating point. It acts through a declared host contract on several published trackers without modifying them. It is evaluated as one frozen policy under confidence shift and on external systems.

A wider family of methods adapts inference itself. Dynamic neural networks adapt their computation to each input [12], and Chameleon re-profiles the configuration of video analytics pipelines over time [13]. These methods learn or profile a policy for each model. Our earlier work, AC-MOT [26], is a training-free controller that sets the detector operating point (score threshold, NMS threshold, and input resolution) online from image and track cues. Such a controller is tied to the detector it was calibrated with, because a change of the detector's score scale

Table 1 Evaluation cells and their role. Development cells were visible while the layer was designed; predeclared cells were run once with the frozen policy.

Data	Detector	Trackers	Role
MOT17 val-half (7 sequences)	published YOLOX-X detections	ByteTrack (2 settings), OC-SORT, BoostTrack, SparseTrack	development
KITTI tracking training (21 sequences)	RT-DETR-L; YOLOv8n	ByteTrack and BoT-SORT (RT-DETR-L); OC-SORT (YOLOv8n)	development (detector transfer)
MOT17 val-half, score transforms	published YOLOX-X, transformed scores	ByteTrack (2 settings), OC-SORT, BoostTrack	development (robustness)
MOT17 val-half	released PD-SORT / Hybrid-SORT detections	PD-SORT, Hybrid-SORT	predeclared external

Table 2 Tracking metrics used in this paper. GT is the number of ground-truth boxes; FP, FN, and IDS are the false positives, missed objects (false negatives), and identity switches, summed over all frames. For IDF1, each ground-truth track is first matched to at most one output track, and IDTP, IDFP, and IDFN are the true-positive, false-positive, and false-negative boxes under this matching. For HOTA, at a localization threshold α, TP_α, FN_α, and FP_α are the matched, missed, and unmatched boxes, and $\mathcal{A}(c)$ is the association accuracy of a matched pair c. All metrics are given in percent.

Metric	Definition	What it measures
MOTA [20]	$\mathrm{MOTA} = 1 - \dfrac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}$	overall error count; reacts strongly to false positives and missed objects
IDF1 [21]	$\mathrm{IDF1} = \dfrac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}$	how long each object keeps the correct identity
HOTA [8]	$\mathrm{HOTA} = \dfrac{1}{19}\sum_\alpha \sqrt{\mathrm{DetA}_\alpha\,\mathrm{AssA}_\alpha}$, $\mathrm{DetA}_\alpha = \dfrac{\lvert\mathrm{TP}_\alpha\rvert}{\lvert\mathrm{TP}_\alpha\rvert + \lvert\mathrm{FN}_\alpha\rvert + \lvert\mathrm{FP}_\alpha\rvert}$, $\mathrm{AssA}_\alpha = \dfrac{1}{\lvert\mathrm{TP}_\alpha\rvert}\sum_{c\in\mathrm{TP}_\alpha}\mathcal{A}(c)$, with $\alpha = 0.05, 0.10, \ldots, 0.95$	balance of detection (DetA) and association (AssA); the main metric of this paper

leaves the image unchanged. SCL reads the score stream instead and acts on the tracker's input and thresholds.

In summary, existing methods either adapt thresholds inside one tracker from the current frame, calibrate scores with labelled data, or learn a policy for one model. A training-free layer that calibrates itself from the detector's own score stream, serves many unmodified trackers, and keeps a well-matched tracker at its own accuracy is missing. SCL is designed to fill this gap. Table 3 summarizes the differences.

Table 3 Summary of the adaptive methods closest to SCL (BT: ByteTrack).

Method	Adapted	Statistics	Where
Adapt. BT [9]	confidence threshold	current frame	inside BT
AdapTrack [10]	matching thresholds	current frame	inside the tracker
ACT [11]	high and low thresholds	current frame	inside the tracker
Küppers [7]	detector scores	labelled data	detector output
Chameleon [13]	pipeline configuration	profiling	analytics pipeline
AC-MOT [26]	detector operating point	image and track cues	before the detector
SCL (this paper)	candidates, scores, thresholds	past frames, no labels	between detector and tracker

4 Method

SCL is a small step that runs once per frame between the detector and the tracker (Fig. 1). In plain terms, it does four things. First, it looks at the detector scores of the last few frames and sorts the candidates into three groups: background, ambiguous, and confident (Sect. 4.2). Second, it uses these groups to decide whether the detector's output is clean or noisy (Sect. 4.3). Third, it acts on that decision: on a clean stream it leaves the tracker's own settings alone, and on a noisy stream it chooses which candidates reach the tracker, gives them new scores, and sets the tracker's thresholds. Fourth, in both cases it can pass a weak candidate that clearly continues an existing track (Sect. 4.4). The tracker then runs as usual on what it receives. The detector and the

tracker are never modified, and Table 5 lists every constant of the layer.

4.1 Problem setting

In each frame t, the detector gives a list of candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$: a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class label c_i for each one, down to a lowest score called the emission floor. The layer does not use the score directly but its logit (log-odds), which spreads out scores close to 0 and 1:

$$z_i = \log \frac{s_i}{1 - s_i}. \tag{1}$$

The tracker, called the host, tells the layer four numbers about itself, the host contract $h = (a, b, \ell, m_0)$. These are simply the settings that its authors published: a is the score a detection needs to be matched to a track in the first step, b is the score it needs to start a new track, ℓ is the lowest score the tracker uses at all, and m_0 is its matching tolerance (one minus the smallest overlap that still counts as a match). A single-stage tracker uses one threshold, so $\ell = \min(a, b)$. A two-stage tracker also uses weaker detections in a second step, so $\ell < \min(a, b)$.

In every frame the layer returns five things: the candidates K_t that the tracker will see, their scores $\tilde{s}$, and the thresholds (a_t, b_t) and tolerance m_t that the tracker will use in this frame:

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t; h\right). \tag{2}$$

Here $Z_{<t}$ are the logits of the earlier frames, $\mathcal{T}_{t-1}$ are the tracks of the previous frame, and g_t is an optional measure of camera motion in frame t. The decision for frame t uses only past frames; frame t is added to the statistics after the decision, and no future frame is ever used.

Four requirements guided the design. The layer needs no training and no labels. One frozen configuration must serve every tracker, detector, and dataset. When the tracker already fits the detector, its input must stay unchanged. And the layer must cost little time next to the detector.

4.2 Self-calibration from the score stream

A detector that suits the scene gives many low scores to background and a separate group of high scores to objects. The layer checks how clearly this happens, without labels. It collects the logits of all candidates of the last $W = 10$ frames and splits them twice with Otsu's method [25], which finds the threshold that best separates a histogram into two groups. The first split, t_1, separates background from foreground. The second split, t_2, divides the foreground into ambiguous and confident candidates. The share of confident candidates in the foreground,

$$\rho_t = \frac{|\{i : z_i \geq t_2\}|}{|\{i : z_i \geq t_1\}|}, \tag{3}$$

tells how cleanly the detector separates objects from clutter. A high ρ_t means that most foreground candidates are confident; a low ρ_t means that many of them are ambiguous. Because the splits are made on logits, they move together with the scores when the whole logit scale is stretched or shifted, for example by a change of temperature, and each group keeps the same candidates.

These groups only make sense if there is a real background group. Some detectors give very few low scores; then the first split falls inside the objects. The layer detects this case, when fewer candidates lie below t_1 than above it, and counts the frame as clean, $\rho_t = 1$. This background check was added after a development case: at the start of some sequences, with about ten candidates per frame, the splits fell at $t_1 \approx 0.6$ and $t_2 \approx 0.93$ in probability terms and gave a wrong noisy decision.

4.3 Intervention regimes

The regime decides how much the layer is allowed to change. A clean stream means that the detector already separates objects from clutter, so the tracker's own settings are kept. A noisy stream means that many candidates are ambiguous, so the layer takes over the choice of candidates and thresholds.

Regime estimate. The stream is clean at frame t if the median $\bar{\rho}_t$ of all ρ values so far (ignoring cold frames) is at least one half, and noisy otherwise. Before the first valid split, for example in the first frame, the regime is called cold. Because the median covers the whole stream so far, the regime describes the detector and the scene, and a short drop in confidence, for example during fast camera motion, does not change it.

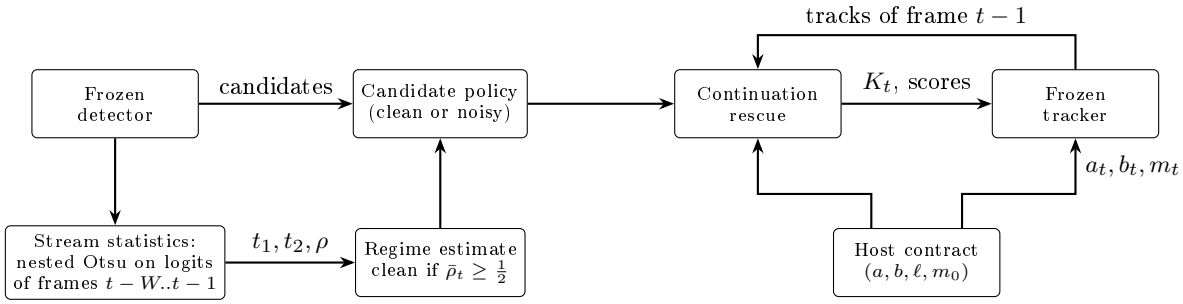


Fig. 1 Self-calibrating control layer (SCL) between a frozen detector and a frozen tracker. The statistics use earlier frames only. The host contract is the tracker's published operating point. The tracker receives the selected candidates, their scores, and its thresholds for the frame.

Cold and clean regimes. The tracker keeps its own thresholds. In the clean regime there is one exception: if a threshold is so high that it would reject the confident group, it is lowered to t_2:

$$a_t = \min(a, \sigma(t_2)), \qquad b_t = \min(b, \sigma(t_2)), \quad (4)$$

where σ is the logistic function, which turns a logit back into a score. Scores are passed unchanged, and only cross-class duplicates (one box proposed with two class labels) are removed. When the confident group already lies above the tracker's thresholds, the tracker therefore receives exactly its normal input.

Example. When the detector scores are halved, no score exceeds 0.5, so a tracker with $a = 0.6$ and $b = 0.7$, such as ByteTrack in its official setting (Table 4), receives no detection that can start a track, so no track is ever created. The Otsu splits are computed from the halved scores themselves, so they move with them. In the clean regime, Eq. (4) then lowers the tracker's thresholds to $\sigma(t_2)$, and the confident candidates reach the tracker again. In the noisy regime, the remap below places them at or above the thresholds of 0.5. Section 7.3 shows the measured effect.

Noisy regime. Here the layer decides which candidates the tracker may use, and for what. Confident candidates ($z \geq t_2$) may start and continue tracks. Ambiguous candidates ($t_1 \leq z < t_2$) may only continue existing tracks. Background candidates ($z < t_1$) are dropped. Each score is replaced by a new one based on its rank u_i among the recent scores of the stream (from 0 for the lowest to 1 for the highest),

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \quad (5)$$

and the tracker's matching and birth thresholds are both set to 0.5, $a_t = b_t = 0.5$. Confident candidates thus get scores of 0.5 and above, and ambiguous ones get scores below 0.5, so they can only reach a tracker's low-score step. Duplicates are removed with the help of the tracks: a weaker candidate that overlaps a stronger one with an intersection over union (IoU) above one half is removed, unless a different track of the previous frame claims it. This keeps people who stand close together in a crowd.

4.4 Continuation rescue and motion rule

A tracker ignores every candidate below its lowest threshold, so a track can be lost when its object scores low for a few frames. This matters most for single-stage trackers, which have no low-score step. The rescue gives such a candidate a second chance, but only when it clearly continues an existing track. Three conditions must hold: the candidate is in the foreground ($z \geq t_1$, or any candidate for a single-stage tracker when there is no background group); it overlaps a track of the previous frame with an IoU of at least one half; and no other usable candidate covers that track. In addition, the tracker must be unable to see the candidate with the thresholds of this frame. The candidate is then passed with its score raised just above the tracker's lowest threshold. For a two-stage tracker, this only concerns candidates below

its low threshold ℓ. For a single-stage tracker, it concerns candidates below the lowest threshold passed for this frame, so the rescue gives it the continuation step it lacks.

When the camera moves fast, a box moves more between two frames, and a fixed overlap requirement can break a correct match. In noisy frames, and only when a motion measure is available, the matching tolerance is therefore widened as the current camera motion grows; r_t is the current motion divided by its median over past frames. The motion is the shift between two consecutive grayscale frames at quarter resolution, found by phase correlation [27] and divided by the image diagonal. The tolerance is

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \qquad (6)$$

4.5 Host contract and adapters

Table 4 lists the contracts used in this paper. Each value is taken from the tracker's published configuration; none is fitted. A small adapter for each tracker family passes the layer's decision to the tracker: the selected candidates and their scores replace the detector output of the frame, and the new thresholds and tolerance replace the tracker's own for that frame. Any post-processing that the authors apply after tracking, such as interpolation or offline linking, is kept. The policy and its constants are the same for every tracker; a tracker enters only through its four contract values, and the adapter only translates the decision, it never makes one.

4.6 Frozen constants

Table 5 lists every constant of the layer; the same values are used for every tracker, detector, and dataset. The thresholds t_1 and t_2 are not constants: they are recomputed in every frame. Each constant has a simple origin. The overlap of one half for duplicates and rescue follows MOT evaluation, where two boxes with IoU above 0.5 cannot both match different objects. The score ranges of Eq. (5) put the two groups on the common thresholds of 0.5 (matching and birth) and 0.1 (low score). The cap of 0.95 keeps a minimum overlap of 0.05 for every match. The window, the motion history, and the remap memory are default values that were kept fixed in all development stages.

Algorithm 1 SCL for frame t

Require: candidates D_t, tracks $\mathcal{T}_{t-1}$, cue g_t, contract (a, b, ℓ, m_0)

$z_i \leftarrow$ logit of s_i for every candidate
if no valid split exists **then**
    regime $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
    $(t_1, t_2) \leftarrow$ nested Otsu on the window
    $\rho_t \leftarrow$ Eq. (3) with the background check
    regime $\leftarrow$ clean if median$(\rho) \geq 0.5$, else noisy
end if
if regime is cold or clean **then**
    remove cross-class duplicates; keep raw scores
    if clean **then** set (a_t, b_t) by Eq. (4)
    end if
else
    remove duplicates with track context
    drop $z < t_1$; remap scores by Eq. (5)
    $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue track continuations (not in cold frames)
$m_t \leftarrow$ Eq. (6) if noisy, else m_0
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
add the logits of frame t to the window; add g_t to the history
store the tracks of frame t after the tracker update

4.7 Per-frame procedure and cost

Algorithm 1 gives the steps for one frame. The work per frame is small: the layer sorts the logits of at most W frames for the two Otsu splits, computes overlaps between candidates and with the tracks of the previous frame, and, when motion is used, computes one phase correlation on a quarter-resolution image. With n candidates per frame and k tracks, the cost is $O(Wn \log(Wn) + n^2 + nk)$ plus the fixed image cost. The layer stores only the logits of the last W frames, the per-frame ρ values, the remap memory, and the last 100 motion values.

Table 4 Host contracts: the operating point that each published tracker declares, as used by the layer. τ is the tracker's own published threshold, which differs between trackers.

Tracker (setting)	Type	assoc a	birth b	low ℓ	match m_0	Decision mapped to
ByteTrack (official MOT17)	two-stage	0.6	0.7	0.1	0.8	thresholds, candidates, scores
ByteTrack (library default)	two-stage	0.25	0.25	0.1	0.8	same
SparseTrack	two-stage	τ	$\tau + 0.1$	0.1	published	same
BoostTrack	two-stage	τ	τ	0.1	1−IoU gate	same
Hybrid-SORT	two-stage	τ	τ	0.1	1−IoU gate	same
OC-SORT	single-stage	0.6	0.6	0.6	0.7	same
PD-SORT	single-stage	τ	τ	τ	1−IoU gate	same

Table 5 Frozen constants of SCL.

Constant	Value
Statistics window W	10 frames
Regime rule	clean if median $\rho \geq 0.5$ over non-cold frames so far
Background check	$\rho = 1$ if fewer candidates below t_1 than above
Noisy score ranges	primary $[0.5, 1)$, extension $[0.1, 0.5)$
Noisy association and birth thresholds	0.5
Remap memory	20 frames, one every 10 frames
Duplicate and rescue IoU	0.5
Rescued score	lowest used threshold $+0.001$
Motion cue	phase correlation, quarter resolution
Motion history, warm-up	100, 5 frames
Tolerance cap	0.95

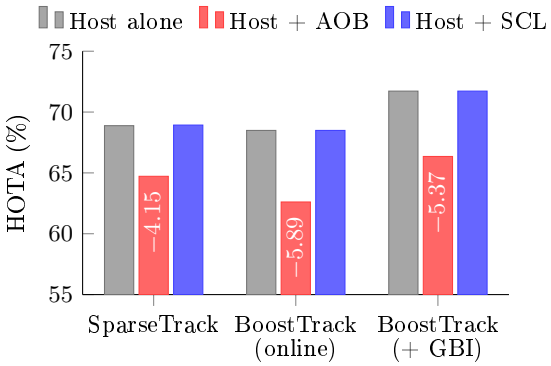


Fig. 2 HOTA on MOT17 val-half of three strong published trackers alone, with AOB, and with SCL. The labels give the change caused by AOB.

5 Development from a Prior Design

5.1 Baseline: always-on banding

SCL was developed from an earlier design of the same research program, called always-on banding (AOB). AOB computed the same nested Otsu bands but treated every stream as noisy. In every frame it applied the bands, removed every weaker candidate that overlapped a stronger one with IoU above one half, and remapped all scores by rank onto fixed ranges. It admitted no candidate at the first frame. AOB was designed for noisy detector streams, and it is the baseline of this section.

Table 6 compares AOB and SCL with the same strong published trackers on MOT17 [14]. AOB lowered the HOTA of SparseTrack by 4.15 points and that of BoostTrack by 5.89 points online and 5.37 points with interpolation, all with intervals below zero. Two causes were found. First, overlap-based duplicate removal deleted occluded people in crowds. Second, the self-calibrated thresholds were stricter than the trackers' own: for SparseTrack and online BoostTrack, false positives fell by 1,630 and 1,351, but false negatives rose by 4,955 and 6,167. On the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left both BoostTrack variants identical to the host. Figure 2 shows the same comparison.

What this stage showed. A layer that intervenes in every frame harms trackers that already fit the detector: three strong published trackers lost between 4.15 and 5.89 HOTA points. Such a layer cannot be placed safely in front of an unknown tracker. The next steps therefore let the layer decide from the score stream whether to intervene at all.

5.2 Design steps

Each step from AOB to SCL was introduced to remove one diagnosed cause and was compared with the step before it on development data (Table 7). The steps, in the order in which they were made, were the following.

Table 6 Baseline comparison of the development stage on MOT17 val-half (published YOLOX-X detections). Host: published tracker alone (our reproduction). Δ: tracker with the prior design AOB or with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero.

Metric	Host	Δ AOB	Δ SCL
SparseTrack [28]			
HOTA	68.88	**−4.15**	+0.06
		[−5.54, −1.66]	[−0.01, +0.25]
MOTA	77.85	**−6.14**	+0.08
		[−8.10, −2.74]	[−0.03, +0.30]
IDF1	81.97	**−4.49**	+0.16
		[−6.19, −1.96]	[−0.02, +0.70]
BoostTrack, online [29]			
HOTA	68.49	**−5.89**	0 (identical)
		[−7.76, −2.87]	
MOTA	75.50	**−8.87**	0 (identical)
		[−12.37, −7.14]	
IDF1	81.41	**−6.22**	0 (identical)
		[−9.33, −2.85]	
BoostTrack, with interpolation [29]			
HOTA	71.72	**−5.37**	0 (identical)
		[−7.30, −2.75]	
MOTA	81.03	**−8.73**	0 (identical)
		[−12.59, −6.50]	
IDF1	84.16	**−5.65**	0 (identical)
		[−8.96, −2.77]	

- *Starting point.* Emulating AOB inside the new framework reproduced the problem: it reduced ByteTrack at floor 0.1 from 67.70 to 56.41 HOTA and OC-SORT from 66.44 to 45.72.
- *Regime gate.* The two causes found in Sect. 5.1 were thresholds stricter than the trackers' own and the removal of occluded people as duplicates. The regime gate adds the regime estimate, the self-limiting clean regime, and the crowd-safe duplicate rule. It restored SparseTrack from 64.72 to 68.93 and BoostTrack from 62.61 to 68.37.
- *Whole-stream regime.* The regime still changed in the middle of some streams. Estimating it from the whole stream removed these changes and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071.
- *Continuation rescue.* Single-stage trackers still lost tracks whose objects scored low for a few frames. The rescue added 0.2 to 0.4 points for OC-SORT.
- *Background check.* A stream with few low-scoring candidates gave a wrong noisy decision (Sect. 4.2). The background check completed SCL: it returned ByteTrack and BoostTrack to output identical to the host and raised OC-SORT to 67.04 and 66.91 at the two floors.

Four variants were rejected on evidence, because they lowered HOTA, added false positives, or let false tracks through on a noisy stream.

What this stage showed. After these steps, the layer kept the strong two-stage trackers at their own accuracy and improved the single-stage OC-SORT. This is the behavior that the frozen configuration was then tested for on new trackers, detectors, and data (Sect. 7).

5.3 Freeze and tests

The final configuration was frozen before any external evaluation. It was recorded in a version-control tag and protected by a lock file that holds cryptographic hashes of the layer, its configuration, and the evaluation runners. Every later run verified the lock before running. Sixty-one automated tests check that the layer uses no future data, that its state resets between sequences, that clean streams pass through unchanged, that the output does not depend on detector, tracker, or sequence names, that affine score transforms leave the bands unchanged where designed, and that the lock is intact.

6 Experimental Setup

The experiments test three expectations from the design. A tracker that already fits its detector should keep its accuracy. A tracker that does not fit, because it lacks a continuation stage or because the detector or its score scale changed, should gain. Both should hold on trackers and data that were not seen during design.

Seven published trackers serve as hosts: ByteTrack [5], BoT-SORT [30], OC-SORT [6], SparseTrack [28], BoostTrack [29], PD-SORT [31], and Hybrid-SORT [32]. Each host uses the authors' code and published configuration, and its contract is given in Table 4.

In every comparison the baseline is the host alone: the published tracker run by us with its published configuration and the same detections. The second arm adds the frozen SCL and changes nothing else. Both arms run in the same environment. Where the authors report their own

Table 7 Design steps from always-on banding (AOB) to SCL on development data (HOTA, %), and variants that were tested and rejected. The floor of ByteTrack and OC-SORT is given in parentheses.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ regime gate	regime statistic, host-anchored clean regime, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream regime	regime from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)
Rejected variant		*Evidence*
no candidates at frame 1		−0.3 to −0.5 HOTA on every MOT17 host
rescue below a two-stage host's low stage		FP +700 to +1000, −0.4 to −0.5 HOTA
rank-remapped scores in clean frames		ByteTrack −0.2 / −1.6 HOTA
host-clipped noisy primary band		KITTI YOLOv8n ByteTrack +1.4 HOTA but uncontrolled false tracks on a noisy aerial stream (26.9 vs 14.9 boxes/frame)

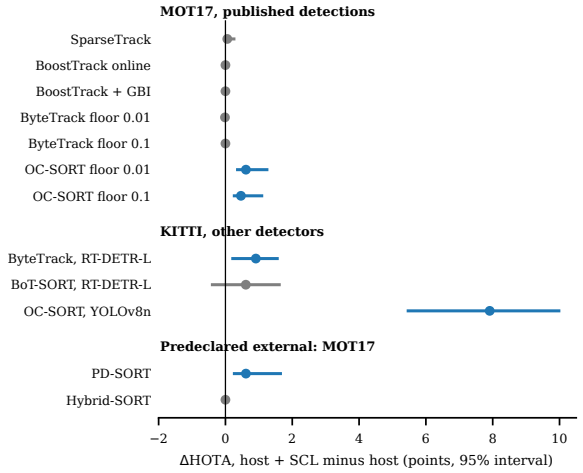


Fig. 3 Difference in HOTA (host with SCL minus host alone) with 95% paired-bootstrap interval for every evaluated cell. Blue: interval above zero; gray: interval contains zero, or identical output.

numbers, these are listed next to our reproduction of the host.

7 Results and Analysis

Figure 3 summarizes the HOTA difference between each host with SCL and the host alone for every evaluated cell. The following subsections give the baseline comparison for each part of the evaluation, in the order of the three expectations of Sect. 6: well-matched trackers, trackers that do not fit their detector, and systems not seen during design.

7.1 Tracker transfer

Table 8 compares the development trackers with their hosts on MOT17. The two-stage trackers with well-matched detections stayed at the accuracy of their hosts. BoostTrack, online and with interpolation, and ByteTrack at floor 0.1 produced identical output. ByteTrack at floor 0.01 changed by −0.014 HOTA and SparseTrack by +0.055. This is the self-limiting behavior of the clean regime (Sect. 4.3), which the prior design did not have (Table 6).

The single-stage OC-SORT improved at both emission floors. Its HOTA rose by 0.61 points ($[+0.36, +1.24]$) at floor 0.01 and by 0.46 points ($[+0.27, +1.09]$) at floor 0.1, and its MOTA by 1.27 and 1.26 points. HOTA was higher on 7 of 7 sequences in both settings (Fig. 4). The gain comes from the continuation rescue. OC-SORT discards every detection below its single threshold, and the layer passes it the foreground candidates that continue a track that would otherwise stay unmatched.

7.2 Detector change

Table 9 reports the KITTI results with the two other detectors. With the noisy RT-DETR-L stream, the two-stage trackers gained in MOTA and IDF1. ByteTrack gained 5.98 MOTA points ($[+1.16, +9.94]$), 3.87 IDF1 points, and 0.91 HOTA points ($[+0.22, +1.55]$). BoT-SORT gained 7.86 MOTA points ($[+2.21, +13.44]$) and 3.10 IDF1 points. With YOLOv8n, the single-stage OC-SORT gained 7.91 HOTA points

Table 8 Baseline comparison of the development trackers on MOT17 val-half (published YOLOX-X detections). Paper: number reported by the tracker's authors. Host: published tracker alone (our reproduction). Δ: host with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. The floor is the lowest detector score emitted.

Metric	Paper	Host	Δ [95% CI]
SparseTrack [28]			
HOTA	69.2	68.88	+0.06 [−0.01, +0.25]
MOTA	76.8	77.85	+0.08 [−0.03, +0.30]
IDF1	81.4	81.97	+0.16 [−0.02, +0.70]
BoostTrack, online [29]			
HOTA	68.37	68.49	0 (identical)
MOTA	75.56	75.50	0 (identical)
IDF1	81.35	81.41	0 (identical)
BoostTrack, + GBI [29]			
HOTA	71.33	71.72	0 (identical)
MOTA	80.55	81.03	0 (identical)
IDF1	83.84	84.16	0 (identical)
ByteTrack, floor 0.01 [5]			
HOTA	–	67.70	−0.014 [−0.058, +0.009]
MOTA	76.6	77.60	+0.06 [−0.15, +0.34]
IDF1	79.3	79.47	−0.031 [−0.135, +0.035]
ByteTrack, floor 0.1 [5]			
HOTA	–	67.70	0 (identical)
MOTA	76.6	77.60	0 (identical)
IDF1	79.3	79.47	0 (identical)
OC-SORT, floor 0.01 [6]			
HOTA	66.5	66.43	**+0.61 [+0.36, +1.24]**
MOTA	74.9	74.67	**+1.27 [+0.17, +3.01]**
IDF1	77.7	78.05	+0.66 [−0.25, +1.60]
OC-SORT, floor 0.1 [6]			
HOTA	66.5	66.44	**+0.46 [+0.27, +1.09]**
MOTA	74.9	74.67	**+1.26 [+0.37, +3.34]**
IDF1	77.7	78.05	**+0.287 [+0.003, +0.838]**

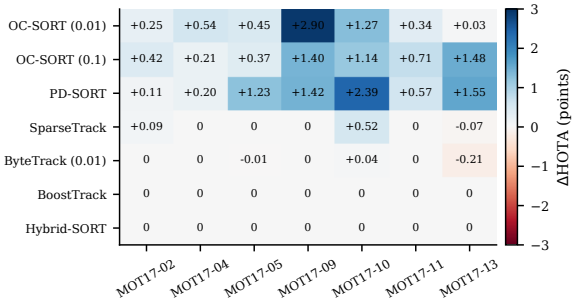


Fig. 4 Per-sequence HOTA differences (host with SCL minus host alone) on MOT17 val-half for the development and predeclared trackers.

([+5.47, +9.98]), 9.40 MOTA points, and 9.60 IDF1 points.

7.3 Detector confidence shift

Table 10 and Fig. 5 give the trackers under the four score transforms. Halving the scores drove

Table 9 Baseline comparison under a detector change on KITTI tracking training (official HOTA, car and pedestrian averaged). Host: published tracker alone with the same detections. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. BoT-SORT uses 20 of the 21 sequences, because the unmodified tracker gives no valid output on one sequence.

Metric	Host	Δ [95% CI]
ByteTrack [5], *RT-DETR-L*		
HOTA	50.73	**+0.91 [+0.22, +1.55]**
MOTA	47.27	**+5.98 [+1.16, +9.94]**
IDF1	64.98	**+3.87 [+1.67, +5.06]**
BoT-SORT [30], *RT-DETR-L*		
HOTA	53.91	+0.61 [−0.39, +1.61]
MOTA	48.25	**+7.86 [+2.21, +13.44]**
IDF1	67.74	**+3.10 [+1.00, +4.57]**
OC-SORT [6], *YOLOv8n*		
HOTA	36.27	**+7.91 [+5.47, +9.98]**
MOTA	33.99	**+9.40 [+1.10, +12.30]**
IDF1	50.46	**+9.60 [+5.61, +12.16]**

ByteTrack (official setting) and OC-SORT to 0 HOTA, because no detection reached their thresholds. With the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60. BoostTrack rose from 61.33 to 64.84 under cubing. With temperature $T = 2$, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points. The library setting of ByteTrack, whose low thresholds already admit the shifted scores, stayed within 0.14 points in all four transforms. Over the 14 conditions, the layer raised HOTA significantly in 7 and lowered it significantly in none.

7.4 External systems

Table 11 gives the two predeclared external systems. PD-SORT [31] was reproduced identically to the authors' released output. With SCL, its HOTA rose by 0.61 points ([+0.27, +1.65]), its MOTA by 1.13 points ([+0.27, +3.03]), and its IDF1 by 0.82 points ([+0.43, +1.94]), with higher HOTA on 7 of 7 sequences (Fig. 4). False negatives fell by 1,291, and false positives rose by 681. PD-SORT is a single-stage tracker, and its gain follows the same mechanism as for OC-SORT; unlike OC-SORT, it was not seen during design. Hybrid-SORT [32], a two-stage tracker, produced identical output, as predicted before the run.

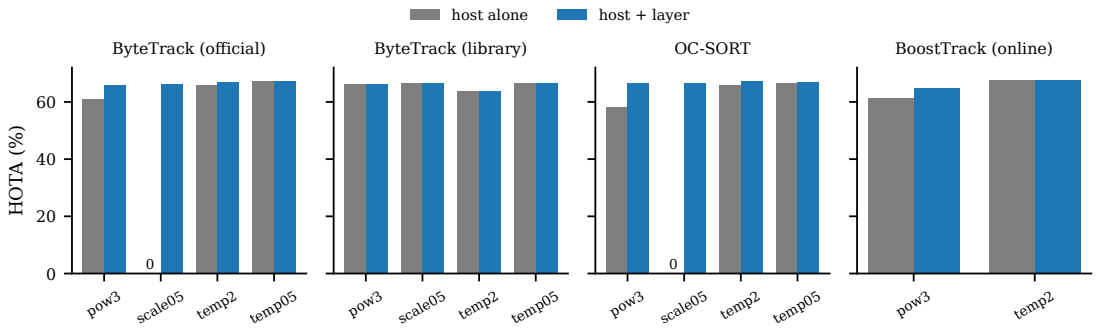


Fig. 5 Detector confidence shift on MOT17 val-half: HOTA of each published tracker with its published thresholds, alone and with SCL, after four monotone transforms of the detector scores (pow3: s^3; scale05: $0.5\,s$; temp2 and temp05: $T = 2$ and $T = 0.5$).

Table 10 Baseline comparison under detector confidence shift on MOT17 val-half (HOTA). The detector scores are transformed before the tracker, whose published thresholds stay fixed; T is a logit temperature, $\sigma(z/T)$. Host: published tracker alone. Δ: with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero. Emission floor 0.01 (BoostTrack 0.1).

Transform	Host	+ SCL	ΔHOTA
ByteTrack, official setting [5]			
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**
$0.5\,s$	0.00	66.17	**+66.17 [+49.32, +75.40]**
$T = 2$	65.84	66.95	**+1.11 [+0.35, +3.87]**
$T = 0.5$	67.30	67.29	−0.009 [−0.051, +0.005]
ByteTrack, library setting [5]			
s^3	66.31	66.31	−0.006 [−0.384, +0.383]
$0.5\,s$	66.61	66.48	−0.14 [−0.66, +0.14]
$T = 2$	63.97	63.97	0 (identical)
$T = 0.5$	66.53	66.52	−0.006 [−0.029, +0.001]
OC-SORT [6]			
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**
$0.5\,s$	0.00	66.72	**+66.72 [+50.32, +75.56]**
$T = 2$	65.99	67.22	**+1.24 [+0.52, +3.42]**
$T = 0.5$	66.61	67.06	+0.45 [−0.01, +0.97]
BoostTrack, online [29]			
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**
$T = 2$	67.59	67.59	0 (identical)

Table 11 Baseline comparison on the predeclared external systems (MOT17 val-half, authors' released detections), each evaluated once with the frozen SCL. Paper: number reported by the authors (for PD-SORT, their released output). Host: published tracker alone (our reproduction). Reproduction class against the authors' number (EXACT: $|\Delta\text{HOTA}| \leq 0.2$; CLOSE: ≤ 1.0 with a documented, untuned cause). Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero.

Metric	Paper	Host	Δ [95% CI]
PD-SORT [31], *EXACT*			
HOTA	68.01	68.01	**+0.61 [+0.27, +1.65]**
MOTA	75.19	75.19	**+1.13 [+0.27, +3.03]**
IDF1	81.03	81.03	**+0.82 [+0.43, +1.94]**
Hybrid-SORT [32], *CLOSE*			
HOTA	67.1	66.70	0 (identical)
MOTA	75.8	75.52	0 (identical)
IDF1	78.0	77.56	0 (identical)

7.5 Latency

Latency was measured after the freeze on one cloud machine with a 4-vCPU Intel Xeon processor at 2.10 GHz, 15 GB of memory, and no GPU. A live YOLOv8n detector (32-bit floats, 736 pixels, batch 1) feeds ByteTrack on three KITTI tracking sequences (0001, 0009, and 0019). Both arms run on the same frames back to back, and the first ten frames of each sequence are excluded, which leaves 2,279 timed frames per arm. The time of the layer includes the motion cue, the per-frame step, and the storage of the tracker output.

Table 12 and Fig. 6 compare the per-frame time of the host pipeline with and without SCL. SCL took 2.95 ms per frame on average and 4.39 ms at the 95th percentile, including the phase-correlation motion cue. The end-to-end time rose from 57.85 to 60.44 ms per frame, by 4.5%. The tracker became faster, from 2.04 to 1.43 ms per frame, because it received fewer candidates. The detector takes most of each frame, about 42 ms, so the layer adds only a small part of the total latency.

7.6 Analysis

When the layer acts. The direction of the effect follows two properties of the pipeline rather than

Table 12 Baseline comparison of the per-frame time on a 4-vCPU Intel Xeon (2.10 GHz, no GPU) with a live YOLOv8n detector (736 pixels) and ByteTrack on three KITTI sequences (2,279 timed frames per arm). Host alone: the same pipeline without SCL. Mean / 95th percentile in milliseconds.

Stage	Host alone	Host + SCL
Frame decode	13.88 / 16.21	13.88 / 16.51
Detector (YOLOv8n, 736 px)	41.91 / 58.41	42.17 / 59.43
SCL	–	2.95 / 4.39
Tracker (ByteTrack)	2.04 / 3.57	1.43 / 2.64
End to end	57.85 / 76.35	60.44 / 80.78

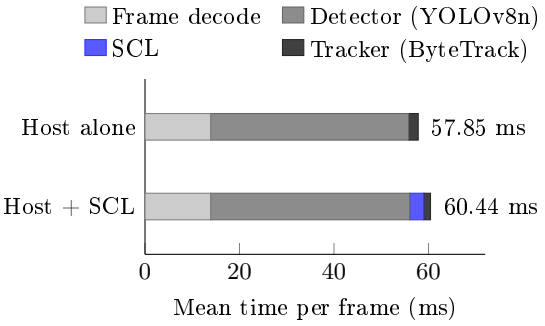


Fig. 6 Mean per-frame time by stage for the host pipeline alone and with SCL (4-vCPU Intel Xeon, no GPU). The labels give the measured end-to-end time.

the dataset: whether the tracker has a continuation stage, and whether the detector's score distribution matches the tracker's thresholds. On MOT17 with well-matched detections, the two-stage trackers BoostTrack, ByteTrack at floor 0.1, and Hybrid-SORT produced output identical to their hosts, while the single-stage OC-SORT and PD-SORT gained HOTA on every sequence. When the stream fits the tracker, the layer stays out of the way, and when the tracker lacks a stage, the layer supplies it.

Self-limiting by design. In the clean regime the layer keeps the tracker's thresholds and scores, so a tracker with a low-score stage and well-matched detections receives its native input. This is the main difference from the prior design AOB, which lowered the HOTA of the same strong trackers by 4.15 to 5.89 points (Table 6). The layer therefore does not need to know in advance whether a tracker needs help; it reads the answer from the score stream.

A continuation stage for single-stage trackers. Single-stage trackers drop every detection below one threshold, so a track can be lost when its object scores low for a few frames. The continuation rescue considers only candidates that overlap a track of the previous frame that no usable candidate covers, so it targets exactly these gaps. OC-SORT and the external PD-SORT gained HOTA on 7 of 7 MOT17 sequences through this stage.

Detector replacement. When a detector is replaced or its confidence scale shifts, the fixed thresholds of a tracker stop matching the scores. The layer re-anchors them from the score stream without labels. Under halved scores it restored two trackers from 0 to about 66 HOTA, and with a different detector it raised the MOTA of two two-stage trackers by 6.0 and 7.9 points. This is the situation the layer was built for: a detector that is swapped, retrained, or quantized while the tracker stays fixed.

Relation to detector-side control. AC-MOT [26] acts before the detector and chooses its operating point from the scene. SCL acts after the detector and adapts the tracker's input to the scores that the detector produces. The two layers address different parts of the pipeline, so they could be used together; combining them is left for future work.

Real-time deployment. The layer needs no training, no GPU, and no access to the detector's internals, and it needs only four contract values from the tracker's configuration. At run time it reads the candidate list, one small grayscale image for the motion cue, and the tracks of the previous frame. On a four-core CPU it takes 2.95 ms per frame, adds 4.5% to the end-to-end time, and makes the tracker itself faster. SCL therefore adds very little latency and is suitable for real-time use. New trackers are added through their published contract values and an adapter, without changes to the policy.

8 Conclusion

This paper presented SCL, a training-free control layer that calibrates itself from a detector's score stream and adapts a frozen tracker through a declared host contract. The layer splits the logits of recent frames with nested Otsu thresholds, estimates whether the stream is clean or noisy, keeps the tracker's operating point on clean streams, and selects candidates, remaps scores,

and sets thresholds on noisy streams. A continuation rescue gives single-stage trackers the stage they lack.

SCL was developed from always-on banding (AOB), an earlier design that intervened in every frame, and each step was compared with that baseline. One frozen configuration was then evaluated on seven published trackers, four detection sources, and two datasets with development and predeclared external roles. It raised the HOTA of OC-SORT and of the predeclared external PD-SORT by 0.61 points, restored trackers whose thresholds no longer matched a shifted confidence scale, raised the MOTA of two-stage trackers with a noisy detector by 6.0 and 7.9 points, and left well-matched two-stage pipelines at their own accuracy. It costs 2.95 ms per frame on a CPU. These results support placing a self-calibrating layer between the detector and the tracker in pipelines where the detector changes while the tracker stays fixed.

References

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7380–7399 (2022) https://doi.org/10.1109/tpami.2021.3119563

[2] Wu, X., Li, W., Hong, D., Tao, R., Du, Q.: Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey. IEEE Geoscience and Remote Sensing Magazine **10**(1), 91–124 (2022) https://doi.org/10.1109/mgrs.2021.3115137

[3] Lu, Y.-R., Sun, D.: Development of Real-Time Unmanned Aerial Vehicle Urban Object Detection System with Federated Learning. Journal of Aerospace Information Systems **21**(7), 547–553 (2024) https://doi.org/10.2514/1.i011378

[4] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple online and realtime tracking. In: 2016 IEEE International Conference on Image Processing (ICIP), pp. 3464–3468 (2016). https://doi.org/10.1109/icip.2016.7533003

[5] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-object Tracking by Associating Every Detection Box. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, pp. 1–21 (2022). https://doi.org/10.1007/978-3-031-20047-2_1

[6] Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9686–9696 (2023). https://doi.org/10.1109/cvpr52729.2023.00934

[7] Küppers, F., Kronenberger, J., Shantia, A., Haselhoff, A.: Multivariate Confidence Calibration for Object Detection. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1322–1330 (2020). https://doi.org/10.1109/cvprw50498.2020.00171

[8] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-object Tracking. International Journal of Computer Vision **129**(2), 548–578 (2021) https://doi.org/10.1007/s11263-020-01375-2

[9] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: 2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS), pp. 370–374 (2023). https://doi.org/10.1109/iccais59597.2023.10382403

[10] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: 2024 IEEE International Conference on Image Processing (ICIP), pp. 2222–2228 (2024). https://doi.org/10.1109/icip51287.2024.10648139

[11] Gwon, D.U., Lee, W.I.: Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking. Electronics **15**(17), 4047 (2026) https://doi.org/10.3390/

electronics15174047

[12] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7436–7456 (2022) https://doi.org/10.1109/tpami.2021.3117837

[13] Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., Stoica, I.: Chameleon: Scalable Adaptation of Video Analytics. In: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, pp. 253–266 (2018). https://doi.org/10.1145/3230543.3230574

[14] Milan, A., Leal-Taixe, L., Reid, I., Roth, S., Schindler, K.: MOT16: A Benchmark for Multi-Object Tracking. arXiv preprint arXiv:1603.00831 (2016)

[15] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. arXiv preprint arXiv:2107.08430 (2021)

[16] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The KITTI vision benchmark suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3354–3361 (2012). https://doi.org/10.1109/cvpr.2012.6248074

[17] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[18] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs Beat YOLOs on Real-time Object Detection. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/cvpr52733.2024.01605

[19] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. International Journal of Computer Vision **129**(4), 845–881 (2021) https://doi.org/10.1007/s11263-020-01393-0

[20] Bernardin, K., Stiefelhagen, R.: Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. EURASIP Journal on Image and Video Processing **2008**, 1–10 (2008) https://doi.org/10.1155/2008/246309

[21] Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C.: Performance Measures and a Data Set for Multi-target, Multi-camera Tracking. In: Computer Vision – ECCV 2016 Workshops. Lecture Notes in Computer Science, pp. 17–35 (2016). https://doi.org/10.1007/978-3-319-48881-3_2

[22] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap, (1993). https://doi.org/10.1007/978-1-4899-4541-9

[23] Wang, C.-Y., Yeh, I.-H., Liao, H.-Y.M.: YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. In: Computer Vision – ECCV 2024. Lecture Notes in Computer Science, pp. 1–21 (2024). https://doi.org/10.1007/978-3-031-72751-1_1

[24] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-End Object Detection with Transformers. In: Computer Vision – ECCV 2020. Lecture Notes in Computer Science, pp. 213–229 (2020). https://doi.org/10.1007/978-3-030-58452-8_13

[25] Otsu, N.: A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics **9**(1), 62–66 (1979) https://doi.org/10.1109/tsmc.1979.4310076

[26] Ismail, A.G., Mohamed, M.S., Mahmoud, T.A.: AC-MOT: Optimization-Guided Adaptive Detector Operating-Point Control for Aerial Multi-Object Tracking. Under review (2026)

[27] Reddy, B.S., Chatterji, B.N.: An FFT-based technique for translation, rotation, and scale-invariant image registration. IEEE Transactions on Image Processing **5**(8), 1266–1271 (1996) https://doi.org/10.1109/83.506761

[28] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. IEEE Transactions on Circuits and Systems for Video Technology **35**(5), 4870–4882 (2025) https://doi.org/10.1109/tcsvt.2024.3524670

[29] Stanojevic, V.D., Todorovic, B.T.: BoostTrack: boosting the similarity measure and detection confidence for improved multiple object tracking. Machine Vision and Applications **35**(3) (2024) https://doi.org/10.1007/s00138-024-01531-5

[30] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv preprint arXiv:2206.14651 (2022)

[31] Wang, Y., Zhang, D., Li, R., Zheng, Z., Li, M.: PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues. IEEE Transactions on Consumer Electronics **71**(1), 165–177 (2025) https://doi.org/10.1109/tce.2025.3541839

[32] Yang, M., Han, G., Yan, B., Zhang, W., Qi, J., Lu, H., Wang, D.: Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking. Proceedings of the AAAI Conference on Artificial Intelligence **38**(7), 6504–6512 (2024) https://doi.org/10.1609/aaai.v38i7.28471